



Think of Kyron as a **building that checks people's IDs**. Over this work we found and locked every weak door — explained here in plain language, each one proven with a test.

5

Weak spots fixed

16/16

Test cases passed

86

Automated tests

0

Regressions

The 5 fixes: 1) Locked the staff data door · 2) Deleted real personal data · 3) Stopped leaking passport numbers · 4) Blocked mass face-fishing · 5) Stopped gesture faking



THE PROBLEM

A hidden admin page listed **everyone’s saved passport details** — and it had **no lock**. Anyone on the internet could open it and read all of it (and even delete everything).

WHAT WE DID

We put a lock on it. It now needs a secret key, and with **no key it is fully disabled**. We also applied the same lock on the public online demo, not just locally.

HOW WE TESTED IT

- No key → door stays shut
- Wrong key → shut
- Correct key → opens
- “Delete everyone” button locked the same way

No key 403

Wrong key 401

Right key 200

✓ PASS



We deleted the real personal data

THE PROBLEM

We discovered **3 real passports** (yours + 2 others), with face data, were actually stored inside the system — kept alive by a hidden file that re-created them.

WHAT WE DID

We found the hidden file, **deleted it**, and wiped the database so nothing re-appears.

HOW WE TESTED IT

- Count the stored people in the database
- Check the hidden re-seed file is gone

People stored **0**

Hidden file **gone**

✓ **PASS**



THE PROBLEM

When the system **recognised a returning face**, it handed back that person's **full passport number**, name, and date of birth — to **anyone**, even a stranger holding a photo.

WHAT WE DID

Now it returns only a first name + a **blurred number** like **PA.....43**, and hides the full name, date of birth, and expiry. Enough to greet “welcome back”, nothing sensitive.

HOW WE TESTED IT

- Confirm full name, DOB, expiry are removed
- Confirm the document number is masked
- Confirm the greeting still works

Passport No. **PA.....43**

Name / DOB / expiry **hidden**

✓ **PASS**



We blocked mass “face-fishing”

THE PROBLEM

An attacker could upload **thousands of faces very fast**, fishing to see which ones match a real saved person — to steal their identity. The old limit failed behind shared internet addresses.

WHAT WE DID

We added a **speed limit**: each person gets only a few tries per minute, plus a **total cap** so mass attempts are rejected — and it now reads the real visitor address behind the proxy.

HOW WE TESTED IT

- Make too many attempts from one client
- Make too many attempts in total

Too many tries **429 blocked**

✓ **PASS**



THE PROBLEM

The hand-gesture step trusted the app to simply say “I did it.” A cheater could **skip the gesture entirely** by sending a fake “done!”, or **replay** an old “done!” message.

WHAT WE DID

The server now issues a **one-time ticket** per gesture (used once, then dead — no replay), and you **cannot do the gesture step** until the earlier, properly-checked steps have passed.

HOW WE TESTED IT

- Try the gesture before the verified step → blocked
- No valid ticket → blocked
- Reuse a ticket (replay) → blocked

Skip ahead **409**

No ticket **401**

Replay **401**

✓ **PASS**

HOW WE PROVED IT

- ✓ **86 automated tests**
The computer re-checks every rule itself — fast and repeatable (one command).
- ✓ **16 documented test cases**
Each with objective, steps, and expected vs actual result — all PASS.
- ✓ **Live checks on the deployed demo**
Poking the real website returns “blocked”: 403 / 401 / 429.
- ✓ **Normal use still works**
A regular person can still complete a verification — nothing broke.

NEXT, FOR PRODUCTION

- **Encrypt stored face data & details (with proper key management)**
- **Add real accounts / login for the main steps (in the production product)**
- **Fully check the gesture on the server (a hand model, not the app’s word)**
- **Bigger-scale rate limiting + activity logs for production**